Degree-sensitive algebraic formula depth reduction with near-linear size refinement

Hervé Fournier, Nutan Limaye, Guillaume Malod, Srikanth Srinivasan, Sébastien Tavenas

doi:10.4230/LIPIcs.CCC.2023.28; CC BY4.0

Editorial note. Original authors: Hervé Fournier, Nutan Limaye, Guillaume Malod, Srikanth Srinivasan, Sébastien Tavenas. arXiv:2302.06984v1 (2023-02-14), https://arxiv.org/abs/2302.06984v1. Creative Commons Attribution4.0, https://creativecommons.org/licenses/by/4.0/.

Editorial note. Selected complete author native main statements/proofs, necessary context, original TikZ code and complete bibliography in standalone layout. Unused frontmatter/packages omitted; no mathematical changes or new proofs. Original source is CC BY4.0.

Allowed standard prerequisites (editorial).

- Standard formula depth reduction and Raz homogenization. Exact prior theorem statements in author preliminaries retained; near-linear Bonet-Buss proof also retained where used.

Here, a *monotone* algebraic formula is one that does not exploit cancellations in any way, and a *non-commutative* formula describes a polynomial expression in a domain where the input variables do not commute when multiplied with each other (formal definitions are given in Section 1). These are both settings in which formula upper bounds are harder to prove, and hence the depth-reduction result in this setting implies the result in the standard setting. It can also be checked that the depth-reduction procedure above preserves other interesting properties of the formula, such as *multilinearity* and *set-multilinearity*.

Using the aforementioned result of Raz that allows us to homogenize algebraic formulas in the low-degree setting, we get the following depth-reduction even for *inhomogeneous* formulas.

Corollary 1. *Let $d = O(\log n)$. Then a homogeneous polynomial P defined on n variables with degree $d \geq 2$ has an algebraic formula of size* $\mathrm{poly}(n)$ *if and only if it has an algebraic formula of depth $O(\log d)$ and size* $\mathrm{poly}(n)$.

1 Preliminaries

Basic notation. Throughout, unless otherwise specified, we work with polynomials over a field $\mathbb{F}$. We will work with the multivariate ring of polynomials $\mathbb{F}[x_1, \ldots, x_n]$ or its *non-commutative* analog $\mathbb{F}\langle x_1, \ldots, x_n \rangle$.

1.1 Algebraic formulas

We start with some brief definitions and results related to algebraic formulas. For much more about this model, see the standard references [SY10, Sap15].

The model. An algebraic formula over the multivariate polynomial ring $\mathbb{F}[x_1, \ldots, x_n]$ is a rooted, directed tree with edges directed towards the root. Leaves are labelled by variables $x_1, \ldots, x_n$ or by the constant 1 and edges by non-zero field constants. Internal nodes (i.e., gates) by $+$ and $\times$ and compute linear combinations (based on the edge weights) or products of their children. We will assume, with loss of generality, that if a node α has for child a leaf labelled by 1, then α is a $+$-gate and that if a $+$-gate α has only children labelled by 1, then α is the output of the formula.[1] A *non-commutative* algebraic formula over the multivariate polynomial ring $\mathbb{F}\langle x_1, \ldots, x_n \rangle$ is defined similarly, with the additional assumption that the children of any $\times$-gate are linearly ordered, and the corresponding product is computed in this order.

Unless explicitly stated, the algebraic formulas we consider have *unbounded* fan-in (i.e., a gate can have any number of inputs). The *size* of F will denote the number of leaves,[2] the *depth* of F the longest leaf-to-root path. The *product-depth* and the *sum-depth* of F are defined to be the maximum number of product gates and sum gates encountered on a leaf-to-root path, respectively.

A *parse tree* of a formula F is a subformula of F which corresponds to the way a monomial is built in the evaluation of F. Parse trees of F can be defined inductively as follows:

- If F has a top $+$-gate, a parse tree of F is obtained by taking a parse tree of one of its children together with the corresponding edge to the root of F;
- If F has a top $\times$-gate, a parse tree of F is obtained by a taking a parse tree of each of its children, together with the incoming edges of the root of F;
- The only parse tree of a leaf is itself.

The polynomial computed by a parse tree is a single monomial, which is equal to the product of the variables labelling its leaves, multiplied by the product of the scalars labelling its edges. The polynomial computed by a formula F is easily seen to be the sum of the monomials computed by all its parse trees.

A formula F is called *monotone* if any monomial computed by a parse tree of F has a non-zero coefficient in the polynomial computed by F.

We now recall some well-known results from the literature regarding algebraic formulas. It should be noted that these results (specifically Theorems 2, 3 and 4 later on) are usually proved in the general setting of commutative formulas. However, it is easy to see that the proofs of these results carry over to the monotone, non-commutative setting without significant change.

[1] This ensures that a formula can compute polynomials with a constant term but forbids using many arithmetic operations just to compute constants.

[2] This is within a constant factor of the number of gates, as long as each gate has fan-in at least 2 each (which is without loss of generality).

Depth-reduction. Classical results [BKM73, Bre74] show that any algebraic formula of small size can be simulated by one of small depth and not much larger size. Formally,

Theorem 2. *Let F be a (non-commutative or commutative) algebraic formula of size s computing a polynomial P. Then there is an algebraic formula F' of size at most* $\mathrm{poly}(s)$ *and depth $\Delta = O(\log s)$ computing P. We may also assume that each gate in F' has fan-in 2. Furthermore, F is homogeneous and/or monotone, then so is F'.*

Homogeneity. Each gate in an algebraic formula has a *syntactic degree* defined in a natural way. Leaves labelled by the constant 1 have syntactic degree 0, leaves labelled with a variable have syntactic degree 1, $\times$-gates have a syntactic degree that is the sum of the syntactic degrees of their children, and $+$-gates have a syntactic degree that is equal to the largest of the syntactic degrees of their children. The syntactic degree of a formula is defined as the syntactic degree of its output. Notice that in a formula the syntactic degree of any gate is bounded by the syntactic degree of the formula.

We will further assume that no gate computes the zero polynomial.

A formula is *homogeneous* if each gate in the formula computes a homogeneous polynomial. Equivalently, in terms of syntactic degrees, this means that all the children of a sum gate have the same syntactic degree.

Raz [Raz13] showed how to convert a possibly inhomogeneous formula F to a homogeneous formula with a relatively small blow-up in size.

Theorem 3. *Let F be a (non-commutative or commutative) algebraic formula of size s and product-depth Δ computing a polynomial P such that all gates in F have fan-in 2. Then there is a* homogeneous *algebraic formula F' of size at most $O\left(s \cdot \binom{\Delta+d+1}{d}\right)$ and product-depth Δ computing P. In particular, if $\Delta = O(\log s)$ and $d = O(\log s)$, then the formula F' has size* $\mathrm{poly}(s)$.

2 Main result

Proof Overview. While the proof of the main result is fairly short and (in our opinion) clean, we add some remarks here to clarify why previous depth-reduction proofs are not applicable in our setting.

The first attempt in proving Theorem 5 would be to try to use the proof strategy behind Theorem 2. Here, we start with a formula F of size s and find a subformula G of size roughly $s/2$ rooted at some gate α of F. It is not hard to show that the polynomial computed by F can be written as

$$F = G \times H_1 + H_2$$

where H_1 and H_2 are also computed by formulas of size $s/2$.[3] We then apply induction to these three subformulas to get the result. Unfortunately, this strategy does not use the degree of the formula at all, and therefore only yields a formula of depth $O(\log s)$.

[3]Here, for simplicity, we are assuming that F is a commutative formula. In the non-commutative setting, we would instead get $F = H_1' G H_1'' + H_2$.

In the homogeneous setting, it is sometimes more natural to do induction on the degree of the underlying formula, in which case G and H_1 (in the decomposition above) would be subformulas of degree roughly $d/2$. We get the following recursion on the worst-case size of the depth-reduced version of F, which we denote by $T(s, d)$

$$T(s, d) \leq T(s_1, d/2) + T(s - s_1, d/2) + T(s - s_1, d)$$

where s_1 denotes the size of G. Unfortunately, in this case, the formulas H_1 and H_2 *could* have size nearly s, resulting in considerable size blow-up. Indeed, when s_1 is much smaller than s (say $s_1 = s^{o(1)}$), the above recursion only yields $T(s, d) = s^{O(\log d)}$, which is a superpolynomial size blow-up.

It may be possible to interleave recursions with respect to size and depth, but we were unable to make this work.

Another possible strategy could be to follow the work of [VSBR83] which produces *circuits* of the required depth. Unfortunately, the proof of [VSBR83] is a memoization procedure, which seems to yield circuits even when applied to a formula F. Turning the resulting circuit of depth $O(\log d)$ into a formula seems to increase the size to $s^{\Omega(\log d)}$.

The approach we take is somewhat more global than the recursive strategies outlined above. Our first motivating example is seemingly the worst-case example for depth-reduction: a *comb* of depth greater than d with alternating sums and products. More formally, a comb computes the following polynomial (up to identifying variables).

$$C(x) = x_1 + (x_2 \times (x_3 + (x_4 \times \cdots)))$$

Note that the above yields a formula of depth greater than d where d is the degree of the underlying polynomial. However, we observe that such a comb actually computes a polynomial with only a few monomials, and hence can be written trivially as a depth-2 $\sum\prod$-formula without much of a size blow-up.

Building on this observation, the overall strategy is to decompose the formula into a top part G, which is a (generalized) comb, whose leaves are subformulas of F to which we will apply the same procedure recursively. We then write G as a $\sum\prod$-formula, and replace its leaves with the depth-reduced versions of the subformulas. This gives the depth-reduced version of F.

The correct definition of G is crucial, and somewhat subtle (at least to us), but with the proper definitions in place, the proof goes through without much trouble.

2.1 Reducing the size blow-up

We note that the above strategy can be easily adapted to yield a small depth formula of size s' that is nearly linear in the size s of the original formula, at the expense of increasing the depth by a large constant.

The proof is nearly identical to the proof of Theorem 5 above. The new ingredient is a near-linear depth-reduction in the setting where there is no bound assumed on the degree of the above formula. More precisely, Bshouty, Cleve and Eberly [BCE95] (see also the work of Bonet and Buss [BB94]) showed the following (we sketch Bonet and Buss' proof in Appendix 3 for completeness).

Theorem 4 (Depth-reduction with near-linear size)**.** *The following holds for any $\varepsilon > 0$. Let F be a (non-commutative or commutative) algebraic formula of size s computing a polynomial P. Then there is an algebraic formula F' of size at most $s^{1+\varepsilon}$ and depth $\Delta = 2^{O(1/\varepsilon)} \cdot \log s$ computing P. We may also assume that each gate in F' has fan-in 2. Furthermore, if F is homogeneous and/or monotone, then so is F'.*

Using the above result, we can prove the following improved version of our depth-reduction.

2.2 Reducing the product fan-ins to 2

It is natural to ask if Theorem 5 can be proved while ensuring that the fan-in of each gate is bounded by 2, as in Theorem 5 and Theorem 4. This is not possible, as a formula of fan-in 2 and depth $O(\log d)$ can only compute polynomials on at most $\text{poly}(d)$ variables, while formulas of size s may have up to s variables. However, this does not rule out reducing the fan-in of the product gates to 2. Indeed, the *circuit* depth-reduction of [VSBR83] does exactly this. We show now that this can also be done for algebraic formulas with bounded syntactic degree.

Theorem 5 (Main Result)**.** *Let F be a homogeneous algebraic formula of size s computing a polynomial P of degree $d \geq 2$. Then P is also computed by a homogeneous formula F' of size $\text{poly}(s)$ and depth $O(\log d)$. Moreover, if F is monotone and/or non-commutative, then so is F'.*

Theorem 6 (Refinement of Theorem 5)**.** *Let F be an algebraic formula of size s and of syntactic degree $d_F \geq 2$. Then P is also computed by a formula F' of size $\text{poly}(s)$ and depth $O(\log d_F)$. Moreover, if F is homogeneous, monotone and/or non-commutative, then so is F'.*

Theorem 7. *Assume that F is a (commutative or non-commutative) formula of size s and syntactic degree $d \geq 1$ computing a polynomial P. Then P is also computed by a formula F'' of size at most $s^{1+\varepsilon}$ and depth $\Delta = 2^{O(1/\varepsilon)} \cdot \log d$. Furthermore, if F is a homogeneous and/or monotone formula, then so is F''.*

Theorem 8. *Let F be a (commutative or non-commutative) algebraic formula F of size s, depth Δ, and syntactic degree $d \geq 1$ computing a polynomial P. Then P can also be computed by a formula F' of size s and depth $\Delta' = O(\Delta + \log d)$ where each product gate of F' has fan-in 2. Furthermore, if F is a homogeneous and/or monotone formula, then so is F'.*

2.3 Proof of Theorem 5

In this section we prove our main result (Theorem 5). We start by showing a simple depth-reduction result for the case of *skew formulas*. A formula is said to be skew if every multiplication gate in it has at most 1 non-trivial child (i.e., a non-leaf node).

We will say that a leaf in a skew-formula is a $+$-leaf if the parent of that leaf is a $+$ gate and a $\times$-leaf otherwise. We show that any skew formula can be converted efficiently into a depth-2 formula, i.e., a $\sum\prod$-formula.

Lemma 9. *Let G be a skew formula with sum-depth δ, wherein all the gates have fan-in 2 and the leaves are labelled by distinct variables. Then the polynomial computed by G is a multilinear polynomial with at most 2^δ monomials. Moreover any variable labelling in G*

- *a $+$-leaf,*
- *or a $\times$-leaf whose sibling is a leaf*

appears in exactly one monomial. We will call them the non-duplicable *variables.*

Proof. We prove this by induction on the depth of the formula. The base case is when the depth is 0 or 1. In both cases, the statement trivially holds.

For the induction case, assume that the depth is at least 2. Suppose $G = G_1 + G_2$. The sum-depths of G_1 and G_2 are at most $\delta - 1$. Let f_1, f_2 be the multilinear polynomials computed by G_1, G_2, respectively. We know that the leaves of G are labelled with distinct variables. Hence, G_1, G_2 have the same property and the variable sets labelling the leaves of G_1 and G_2 are disjoint. The depths of G_1 and G_2 are strictly smaller than the depth of G. By the induction hypothesis we have that f_1 and f_2 have at most $2^{\delta-1}$ monomials. Moreover, any non-duplicable variable in G_1 appears in at most one monomial in f_1. Similarly, any non-duplicable variable in G_2 appears in at most one monomial in f_2. Hence, the polynomial computed by G, i.e., $f_1 + f_2$, has at most 2^δ monomials and each non-duplicable variable appears in at most one monomial in it.

Suppose the top gate of G is a $\times$ gate. As G is a skew formula it is either $x \times G_1$ or $G_1 \times x$, where x is a variable. In particular since the depth of G is at least two, the variable x is duplicable. Let f_1 be the multilinear polynomial computed by G_1. By our assumption, the variable x does not appear in f_1. The depth of G_1 is strictly smaller than G. By the induction hypothesis we have that f_1 has at most 2^δ monomials and any non-duplicable variable in G_1 appears in at most one monomial. As x can distribute over the monomials of f_1, we have that the polynomial computed by G is multilinear with at most 2^δ monomials and any non-duplicable variable appears in at most one monomial.

(Note that this can also be seen using parse trees, since the polynomial computed is the sum of the monomials computed by the parse trees. The multilinearity is obvious. Note that parse trees do not really "branch" at multiplication gates because of the skewness and are therefore combs. To build a parse tree starting from the root we will have two choices for each addition gate we encounter on the path, and there are at most δ, so we get at most 2^δ parse trees. The only parse tree containing a given non-duplicable variable is defined by the path from the root to this leaf.) □

If a formula is homogeneous, it implies that for any gate α, the degree of the polynomial computed by α coincides with the syntactic degree d_α of α. Based on this remark, below we prove a stronger statement than Theorem 5. Specifically, Theorem 5 is stated for homogeneous formulas. But here, we show a depth-reduction for formulas for which the syntactic degree is small.

Proof. Let us start with a formula obtained from F after applying Theorem 2. That is, we will assume that we have a formula of size poly(s) such each gate

in it has fan-in 2, sum-depth and product-depth are bounded by $O(\log s)$. For notational simplicity, from now on, F will refer to this new formula.

Let δ be a positive integer. For a formula G of syntactic degree $d_G \geq 1$ and sum-depth $\Delta(G)$ we define a potential function $\phi_\delta(G)$ as follows.

$$\begin{cases} \phi_{\delta,1}(G) = \lceil \log(d_G) \rceil \\ \phi_{\delta,2}(G) = \lceil \Delta(G)/\delta \rceil \end{cases}$$

and let

$$\phi_\delta(G) = \phi_{\delta,1}(G) + \phi_{\delta,2}(G).$$

We will show that the potential function bounds the depth of the depth-reduced formula that we will construct. We will also use it to bound the size of the resulting formula. Specifically, we prove the following lemma.

Lemma 10. *Let δ be a positive integer. Any formula F of fan-in 2, syntactic degree $d \geq 1$, sum-depth Δ, and size s can be parallelized into a formula F' (of arbitrary fan-in) of product-depth at most $\phi_\delta(F)$ and size at most $s \cdot 2^{\delta \log(d)}$. Further, if F is homogeneous, monotone and/or non-commutative, so is F'.*

Since we ensured that the sum-depth is bounded by $O(\log s)$, taking $\delta = \left\lceil \frac{\log s}{\log d} \right\rceil$ and applying Lemma 10, we get that the final formula F' has size at most $\text{poly}(s)$ and product-depth at most

$$\phi_\delta(F) = O\left(\lceil \log d \rceil + \left\lceil \log(s) \frac{\log d}{\log s} \right\rceil \right) = O(\log d).$$

By collapsing sum gates that feed into other sum gates, we see that the depth of F' can be assumed to be at most twice its product-depth, which is $O(\log d)$. This thus finishes the proof of the theorem. □

We now prove Lemma 10.

Proof of Lemma 10. For any gate α in F, let F_α denote the subformula rooted at α and d_α be its syntactic degree. We do the proof by induction on $\phi_\delta(F)$.

The base case $\phi_\delta(F) = 0$ trivially holds. Consider the following set of gates of F:

$$\mathcal{A} = \left\{ \alpha \mid \alpha \text{ is a not a leaf labelled 1 and } \phi_\delta(F_\alpha) < \phi_\delta(F) = \phi_\delta(F_{\text{parent}(\alpha)}) \right\}.$$

For any gate α in $\mathcal{A}$, the induction hypothesis tells us that we can construct a formula F'_α of product-depth at most $\phi_\delta(F) - 1$ and size at most $s_\alpha \cdot 2^{\delta \log(d_\alpha)}$ computing the same polynomial as F_α.

Let us consider the formula G obtained by replacing these gates from $\mathcal{A}$ in F by leaves (labelled with distinct variables). Notice that for a product-gate β in F, at most one of its children has syntactic degree larger than $d_\beta/2$, where d_β is the syntactic degree of β. Consequently, G is a skew formula. The other child of β is a $\times$-leaf in G. Moreover, G has sum-depth at most δ (since $\phi_{\delta,2}$ strictly decreases for gates below).

Hence, we can use Lemma 9 to simplify G: we get that the polynomial computed by G is a multilinear polynomial in its leaves and has 2^δ monomials. We can then write G as a $\sum\prod$-formula G' such that each duplicable gate appears in at most 2^δ monomials and each non-duplicable gate in at most 1.

The new formula F' for F is obtained from G' by replacing each variable leaf, which corresponds to some gate α in F, by its depth-reduced version F'_α constructed using the induction hypothesis above.

The product-depth of F' is bounded by the product-depth of the gates $\alpha \in \mathcal{A}$ plus the product-depth of G, which is equal to 1 after rewriting it as a $\sum\prod$-formula. That is, the product-depth is at most $(\phi_\delta(F) - 1) + 1 = \phi_\delta(F)$. By construction if G does not contain leaves labelled by 1, then it is also the case for G', otherwise, G' still has at most one such leaf. The size of the resulting formula is bounded by

$$\sum_{\alpha \text{ non-duplicable}} \left(s_\alpha \cdot 2^{\delta \log(d_\alpha)}\right) + \left(2^\delta \cdot \sum_{\alpha \text{ duplicable}} \left(s_\alpha \cdot 2^{\delta \log(d_\alpha)}\right)\right) + \mathbb{1}_{\text{G' has a constant leaf}}.$$

Notice that if α is duplicable, it must be a $\times$-leaf with its sibling β not a leaf. This means that $d_\alpha \leq d_\beta \leq d$ since the syntactic degree is maximal at the root. Hence $d_\alpha \leq d/2$ so the contribution of duplicable gates is bounded by

$$\sum_{\alpha \text{ duplicable}} 2^\delta \left(s_\alpha \cdot 2^{\delta(\log(d)-1)}\right) \leq \sum_{\alpha \text{ duplicable}} \left(s_\alpha \cdot 2^{\delta \log(d)}\right).$$

The contribution of non-duplicable gates is bounded by

$$\sum_{\alpha \text{ non-duplicable}} s_\alpha \cdot 2^{\delta \log d}$$

since any gate in F has syntactic degree at most d. By the choice of $\mathcal{A}$, the subformulas F_α are disjoint so $\sum_{\alpha \in \mathcal{A}} s_\alpha \leq s$, with strict inequality if G' has a constant leaf. Hence the size of F' is bounded by $s \cdot 2^{\delta \log(d)}$.

Finally, it is straightforward to verify that the construction preserves homogeneity, monotonicity and/or non-commutativity. □

Remark 11. *It is easy to note that our depth reduction procedure does not increase the syntactic degree of the formula.*

We also observe that putting Theorem 5 together with Theorems 2 and 3 immediately implies Corollary 1.

Proof of Corollary 1. Given a (possibly inhomogeneous) formula F of size $s = \text{poly}(n)$ computing a polynomial of degree $d = O(\log n)$, we first apply standard depth-reduction (Theorem 2) to get an equivalent formula F_1 of size $s_1 = \text{poly}(n)$ and depth $\Delta_1 = O(\log n)$ where each gate has fan-in 2. Applying Raz's homogenization theorem (Theorem 3) to F_1 yields an equivalent *homogeneous* formula F_2 of size $s_2 = \text{poly}(n)$ and depth $\Delta_2 = O(\log n)$. We can now apply Theorem 5 to F_2 to get an equivalent formula F' of size $s' = \text{poly}(n)$ and depth $\Delta' = O(\log d)$. □

Proof. If $d^{4/\varepsilon} \geq s$, then the depth-reduction of Theorem 4 already gives a satisfactory solution. Indeed the size is bounded by $s^{1+\varepsilon}$ and the depth is bounded by

$$2^{O(1/\varepsilon)} \log s \leq \frac{4}{\varepsilon} 2^{O(1/\varepsilon)} \log d \leq 2^{O(1/\varepsilon)} \log d.$$

So we assume that $s > d^{4/\varepsilon}$. By first applying Theorem 4 (with $\varepsilon/2$ instead of ε), we obtain a formula F' of size at most $s^{1+\varepsilon/2}$, depth $\Delta' = 2^{O(1/\varepsilon)} \cdot \log s$, and fan-in 2 computing P.

Now, we apply Lemma 10, while setting $\delta = \lfloor (\varepsilon \log s)/(2 \log d) \rfloor$. Notice that since $\varepsilon \log s > 4 \log d$, it ensures that $\delta > (\varepsilon \log s)/(4 \log d) > 1$.

Then the strategy produces a formula F'' of product-depth at most

$$\phi_\delta(F') \leq \phi_{\delta,1}(F') + \phi_{\delta,2}(F') < \lceil \log d \rceil + 2^{O(1/\varepsilon)} \log s \frac{4 \log d}{\varepsilon \log s} = 2^{O(1/\varepsilon)} \log d$$

and size at most

$$s^{1+\varepsilon/2} \cdot 2^{\delta \log d} \leq s^{1+\varepsilon/2} s^{\varepsilon/2}.$$

This proves the theorem. □

Proof. It suffices to prove a weaker version of the above theorem where each product gate has fan-in at most 3. We can then replace each of the products of fan-in 3 by a tree of product gates of fan-in 2 and size 3. This has the effect of increasing the depth at most by a factor of 2, which does not affect the overall result.

So we will prove this slightly weaker version. In this setting, we will aim for a depth $\Delta' = \Delta + \log d$.

This is done by induction on the depth Δ of the formula. The case of $\Delta = 0$ is trivial. Let F be a formula of depth $\Delta > 0$ and syntactic degree d.

Assume that the output gate of F is a sum gate, and $F_1, \ldots, F_t$ are the subformulas of F of depth $\Delta - 1$. By definition, each F_i has syntactic degree at most d. Applying the induction hypothesis to each of the F_i yields a formula F'_i with product gates of fan-in at most 3. The formula F' can then be defined as the sum of these formulas.

Now we come to the main case, which is when the output gate of F is a product gate. Assume that $F_1, \ldots, F_t$ are the subformulas of F of depth $\Delta - 1$ in the order[4] that they appear in F. Let d_i denote the syntactic degree of F_i. Define

$$m = \min\{j \mid \sum_{i=1}^{j} d_i \geq d/2\}.$$

Let F_ℓ be the formula obtained from F by keeping only the subformulas $F_1, \ldots, F_{m-1}$, and F_r be the formula obtained by keeping only $F_{m+1}, \ldots, F_t$. We use the induction hypothesis on F_ℓ, F_m, and F_r to get formulas F'_ℓ, F'_m and F'_r. Finally, we set

$$F' = F'_\ell \times F'_m \times F'_r.$$

The size[5] of F' is the sum of the sizes of F'_ℓ, F'_m and F'_r, which is at most s by the induction hypothesis. Let Δ'_ℓ, Δ'_m and Δ'_r denote the depths of F'_ℓ, F'_m and F'_r respectively. The depth of F' is

$$1+\max\{\Delta'_\ell, \Delta'_m, \Delta'_r\} \leq 1+\max\{\Delta+\log(d/2), \Delta-1+\log d, \Delta+\log(d/2)\} = \Delta+\log d$$

where the second inequality uses the induction hypothesis, and the fact that F_ℓ and F_r have syntactic degree at most $d/2$ and F_m has depth at most $\Delta - 1$. □

[4] The order is important in the non-commutative setting.

[5] Recall that the size of a formula is the number of its leaves.

3 The Bonet-Buss depth-reduction

In this section, we give a proof of Theorem 4. This is essentially the same proof as in the work of Bonet and Buss [BB94], but since the results in that paper are only stated for Boolean formulas, we give the proof here for completeness.

We show that for some large enough absolute constant $C > 0$, any polynomial P computed by a formula F of size s can also be computed by a formula F' of size at most $s^{1+\varepsilon}$, depth at most $2^{O(1/\varepsilon)} \cdot \log s$, and fan-in 2. Further, if F is homogeneous/monotone/non-commutative, then so is F'. This latter claim will follow directly from the construction.

We assume here that each gate in F has fan-in 2 to begin with. We can make this modification to F in the beginning at the expense of increasing the depth, and without increasing the size.

We prove the claim by induction on the size s of the formula. Let $T(s)$ and $D(s)$ denote the maximum size and depth (respectively) of F' thus obtained, assuming that F has size at most s. Let $k = 2^{C/\varepsilon}$ where $C > 0$ is an absolute constant we will choose below. We assume throughout that $\varepsilon < 1$, which is without loss of generality.

The base case of the formula is when $s \leq k$, in which case the claim is trivial, as any formula of size s has depth at most s, which in this case is at most $2^{C/\varepsilon}$.

Now, assume that $s > k$. In this case, we find a gate α such that the size of the subformula F_α rooted at α has size at least $s - s/k$, while the children β and γ of α do not satisfy this property. It is easy to observe that there is a unique α with this property. Let $*$ denote the operation (either $+$ or $\times$) labelling α.

We replace α by a fresh variable y in F to get a formula F_y. Note that F_y has size at most $s_1 = \frac{2s}{k}$. Since F_y has at most one occurrence of y, we can write

$$F_y = A \cdot y \cdot B + C$$

where A is the product of all subformulas that multiply y on the left in F along the path to the output, B is similarly the product of all subformulas that multiply y on the right, and C is the polynomial computed by F_y when we set y to 0. Clearly, A, B and C have formulas F_A, F_B and F_C of size at most s_1. Finally, we get the formula F' as in Figure 1.

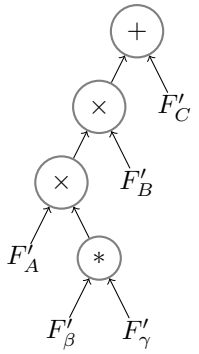


Figure 1: Constructing F'

Here, $F'_A, F'_B, F'_C, F'_\beta, F'_\gamma$ are the formulas obtained by recursively applying the same procedure to $F_A, F_B, F_C, F_\beta, F_\gamma$.

We can bound the size and depth of the depth-reduced formula by induction. We have

$$D(s) \leq D(s - s/k) + O(1)$$

leading easily to an overall depth bound of $O(k \log s) = 2^{O(1/\varepsilon)} \cdot \log s$ for any choice of the constant C.

For the size, we have the following recursion.

$$T(s) \leq \max_{\substack{s_\beta, s_\gamma, s_1: \\ s_\beta, s_\gamma \leq s(1-1/k) \\ s_1 \leq 2s/k \\ s_\beta + s_\gamma + s_1 \leq s}} T(s_\beta) + T(s_\gamma) + 3T(s_1)$$

where s_β and s_γ are the sizes of F_β and F_γ respectively. We now use induction to bound $T(s)$ as follows. (We omit the conditions on s_β, s_γ and s_1 for notational simplicity.)

$$\begin{aligned} T(s) &\leq \max_{s_\beta, s_\gamma, s_1} s_\beta^{1+\varepsilon} + s_\gamma^{1+\varepsilon} + 3s_1^{1+\varepsilon} \\ &\leq \left(s\left(1 - \frac{1}{k}\right)\right)^{1+\varepsilon} + \left(\frac{s}{k}\right)^{1+\varepsilon} + 3\left(\frac{2s}{k}\right)^{1+\varepsilon} \end{aligned}$$

where we used the fact that $s_1 \leq 2s/k$ and the fact that, using the convexity of the map $x \mapsto x^{1+\varepsilon}$, the function $s_\beta^{1+\varepsilon} + s_\gamma^{1+\varepsilon}$ is maximized when $\max\{s_\beta, s_\gamma\} = s - s/k$, meaning that $\min\{s_\beta, s_\gamma\} \leq s/k$.

Continuing the computation, we get

$$T(s) \leq s^{1+\varepsilon}\left(\left(1 - \frac{1}{k}\right) + \frac{1}{k^{1+\varepsilon}} + 3\frac{4}{k^{1+\varepsilon}}\right) \leq s^{1+\varepsilon} \cdot \left(1 - \frac{1}{k} + \frac{C'}{k^{1+\varepsilon}}\right)$$

for some large enough absolute constant C'. Setting $k = 2^{C/\varepsilon}$ for a large enough absolute constant C gives us $T(s) \leq s^{1+\varepsilon}$, proving the inductive claim.

References

[AV08] Manindra Agrawal and V. Vinay. Arithmetic circuits: A chasm at depth four. In *proceedings of Foundations of Computer Science (FOCS)*, pages 67–75, 2008.

[B00] Peter Bürgisser. Cook's versus Valiant's hypothesis. *Theoretical Computer Science*, 235(1):71–88, 2000.

[BB94] Maria Luisa Bonet and Samuel R. Buss. Size-depth tradeoffs for boolean formulae. *Information Processing Letters*, 49(3):151 – 155, 1994.

[BCE95] Nader H. Bshouty, Richard Cleve, and Wayne Eberly. Size-depth tradeoffs for algebraic formulas. *SIAM J. Comput.*, 24(4):682–705, 1995.

[BKM73] R. Brent, D. Kuck, and K. Maruyama. The parallel evaluation of arithmetic expressions without division. *IEEE Transactions on Computers*, C-22(5):532–534, 1973.

[Bre74] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, April 1974.

[CKSV16] Suryajith Chillara, Mrinal Kumar, Ramprasad Saptharishi, and V. Vinay. The chasm at depth four, and tensor rank : Old results, new insights. *CoRR*, abs/1606.04200, 2016.

[CLS18] Suryajith Chillara, Nutan Limaye, and Srikanth Srinivasan. Small-depth multilinear formula lower bounds for iterated matrix multiplication, with applications. In *STACS*, volume 96 of *LIPIcs*, pages 21:1–21:15. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2018.

[GKKS16] Ankit Gupta, Pritish Kamath, Neeraj Kayal, and Ramprasad Saptharishi. Arithmetic circuits: A chasm at depth 3. *SIAM Journal of Computing*, 45(3):1064–1079, 2016.

[HY11] Pavel Hrubeš and Amir Yehudayoff. Homogeneous formulas and symmetric polynomials. *Comput. Complexity*, 20(3):559–578, 2011.

[KdOS19] Mrinal Kumar, Rafael Mendes de Oliveira, and Ramprasad Saptharishi. Towards optimal depth reductions for syntactically multilinear circuits. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, *46th International Colloquium on Automata, Languages, and Programming, ICALP 2019, July 9-12, 2019, Patras, Greece*, volume 132 of *LIPIcs*, pages 78:1–78:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.

[Koi12] Pascal Koiran. Arithmetic circuits: The chasm at depth four gets wider. *Theor. Comput. Sci.*, 448:56–65, 2012.

[Kos86] S Rao Kosaraju. Parallel evaluation of division-free arithmetic equations. In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, pages 231–239, 1986.

[KSS14] Neeraj Kayal, Chandan Saha, and Ramprasad Saptharishi. A super-polynomial lower bound for regular arithmetic formulas. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 146–153. ACM, 2014.

[LST21a] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. Super-polynomial lower bounds against low-depth algebraic circuits. *Electron. Colloquium Comput. Complex.*, TR21-081, 2021.

[LST21b] Nutan Limaye, Srikanth Srinivasan, and Sébastien Tavenas. Super-polynomial lower bounds against low-depth algebraic circuits. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, pages 804–814. IEEE, 2021.

[Nis91] Noam Nisan. Lower bounds for non-commutative computation. In *Proceedings of the twenty-third annual ACM symposium on Theory of computing*, pages 410–418, 1991.

[NW97] Noam Nisan and Avi Wigderson. Lower bounds on arithmetic circuits via partial derivatives. *Computational Complexity*, 6(3):217–234, 1997.

[PM75] Franco P. Preparata and David E. Muller. The time required to evaluate division-free arithmetic expressions. *Inf. Process. Lett.*, 3(5):144–146, 1975.

[PM76] Franco P. Preparata and David E. Muller. Efficient parallel evaluation of boolean expression. *IEEE Trans. Computers*, 25(5):548–549, 1976.

[Raz06] Ran Raz. Separation of multilinear circuit and formula size. *Theory of Computing*, 2(1):121–135, 2006.

[Raz13] Ran Raz. Tensor-rank and lower bounds for arithmetic formulas. *Journal of the ACM*, 60(6):40:1–40:15, 2013.

[RY08] Ran Raz and Amir Yehudayoff. Balancing syntactically multilinear arithmetic circuits. *Computational Complexity*, 17(4):515–535, 2008.

[Sap15] Ramprasad Saptharishi. A survey of lower bounds in arithmetic circuit complexity. Github survey, 2015.

[Spi71] Philip M. Spira. On time hardware complexity tradeoffs for Boolean functions. In Shu Lin, editor, *Proceedings of the Fourth Hawaii International Conference on System Sciences*, pages 525–527. Western Periodicals Company, North Hollywood, California, 1971.

[SS80] Eli Shamir and Marc Snir. On the depth complexity of formulas. *Math. Syst. Theory*, 13:301–322, 1980.

[SW01] Amir Shpilka and Avi Wigderson. Depth-3 arithmetic circuits over fields of characteristic zero. *Computational Complexity*, 10(1):1–27, 2001.

[SY10] Amir Shpilka and Amir Yehudayoff. Arithmetic circuits: A survey of recent results and open questions. *Foundations and Trends in Theoretical Computer Science*, 5:207–388, March 2010.

[Tav15] Sébastien Tavenas. Improved bounds for reduction to depth 4 and depth 3. *Information and Computation*, 240:2–11, 2015.

[TLS22] Sébastien Tavenas, Nutan Limaye, and Srikanth Srinivasan. Set-multilinear and non-commutative formula lower bounds for iterated matrix multiplication. In *STOC*, pages 416–425. ACM, 2022.

[VSBR83] Leslie G. Valiant, Sven Skyum, Stuart J. Berkowitz, and Charles Rackoff. Fast Parallel Computation of Polynomials Using Few Processors. *SIAM Journal of Computing*, 12(4):641–644, 1983.

[YK68] SV Yablonskii and VP Kozyrev. Mathematical problems of cybernetics. *Information Materials of Scientific Council of Akad. Nauk SSSR on Complex Problem "Kibernetika*, 19:3–15, 1968.